Compiler Verification in oksolc: A Source-Based Review of Proofs and Boundaries

Repository review
Snapshot: 4 November 2026
Public preview proof sources

***Abstract*—The oksolc Lean development combines source-bound implementation models, restricted Zig AIR translation, and certificates for selected compiler artifacts. This is a sound architectural choice provided that model adequacy, source admission, host contracts, and unclosed compiler premises remain explicit. It does not currently establish correctness of the complete Solidity-to-EVM compiler. Substantial proof sources cover algebraic rewrites, normalization, ownership, finite recursive optimizer executions, typed memory, and restricted deployment/runtime artifacts. The main implementation gap is composition from actual compiler initialization and native data structures to the endpoint's preconditions. A concrete remaining adapter limitation rejects anonymous empty byte slices in a captured hash-update call. We state the relevant obligations, inventory representative proof endpoints, and recommend prioritizing artifact validation for semantic breadth while bounding native proof work by removal of specific endpoint premises. This review inspected source only: no Lean build, kernel replay, compiler execution, or historical acceptance run was performed.**

I. Scope and interpretation of evidence

The review concerns the committed development in `lean/`, with supporting inspection of its Python extractors, `test/lean/` certificate drivers, compiler sources, and verification documentation. The accompanying proof snapshot contains 1,440 Lean source files including the Lake file, 172,081 lines, and 404 files whose names contain "Test." These are maintenance measurements, not measures of verified compiler coverage. The main distribution is 1,147 files in `Compiler/Extraction`, 226 in `Compiler/ZigAir`, 38 in `Compiler/Validation`, and four in `Compiler/Algebraic`; shared language, EVM, interaction, and audit modules reside in `Contract/`.

The review traced representative top-level theorem statements, their principal predicates, interpreters, extraction boundaries, and artifact acceptance code. It is a global architectural and implementation-quality review, not a claim to have independently checked every proof term or every line of this corpus. Throughout this paper, "proof present" means a theorem and proof body exist in the inspected source. It does not mean that this review freshly elaborated that theorem or reproduced a production certificate. Current acceptance requires evidence tied to the exact public commit; it is separate from the source-supported statements in this paper.

The project pins Lean 4.29.0 and enables warnings as errors. A lexical scan of the Lean sources found no occurrences of `sorry` or `native_decide`, no explicit axiom declarations, and no declarations beginning with `unsafe def` or `partial def`. This is useful hygiene, but cannot replace a compiled dependency audit and kernel replay. The dependency auditor permits only `propext`, `Classical.choice`, and `Quot.sound`; it also rejects unsafe, partial, missing, and non-closed dependencies [2].

The coverage map declares 34 obligations: 13 closed, ten partial, and eleven open [1]. These are heterogeneous scoped obligations, not a percentage of compiler correctness. In particular, a closed artifact route means that its restricted certificate mechanism has proofs; it does not mean that every current compiler output is accepted.

II. Compiler notation and intended correctness

A. Requests, stages, and artifacts

Let s denote the exact Solidity source collection, including source names, and let θ contain the language version, EVM fork, optimizer settings, metadata policy, library bindings, and other admitted compilation settings. Write the implemented compiler as

$$C_\theta(s) \in \mathrm{Result}(\mathcal{A}, \mathcal{E}_C), \qquad a = (b_{\mathrm{init}}, b_{\mathrm{run}}, \mu) \in \mathcal{A},$$

where b_{init} and b_{run} are exact creation and runtime bytes, μ is auxiliary output, and $\mathcal{E}_C$ contains compiler failures. Compiler failure is distinct from a successfully compiled program reverting at runtime. A useful conceptual decomposition is

$$s \xrightarrow{P} A \xrightarrow{T} A_t \xrightarrow{L} Y \xrightarrow{O_Y} Y' \xrightarrow{B} E \xrightarrow{O_E} E' \xrightarrow{J} a.$$

Here P, T, L, O_Y, B, O_E, J denote parsing, semantic analysis, lowering to Yul, Yul optimization, backend lowering, assembly optimization, and assembly/linking. This is notation for proof interfaces, not a claim that all compiler modes execute this identical sequence. The repository separately tracks these stages, request orchestration, and their currently open implementation obligations [1].

B. Observable behavior

Let ω be a persistent world, τ transaction-local state, κ a call or creation context, x calldata and incoming value, and g available execution gas. An observation may contain

$$o = (\mathrm{status}, \mathrm{bytes}, \omega', \tau', \mathrm{logs}, \mathrm{interactions}, g').$$

The intended comparison must specify which components are visible and what transaction admission has already occurred.

The current artifact fragments do not implement arbitrary logs or external calls. Their absence is part of the fragment, not a proof about general effectful Solidity.

There is no universal claim that two compiler outputs consume equal gas. For admitted code without gas introspection or external calls, the certificate can establish a source functional outcome together with the exact target execution cost. It then derives the target observation at every gas budget. Storage execution additionally tracks warm accesses, original/current slots, refund deltas, static mode, and an execution threshold for the SSTORE sentry. This differs from inventing an independent source-level gas cost and proving cost equality with it.

Obligation 1 (Source-to-bytecode endpoint)**.** *For a declared source/settings domain D, observation policy $\mathcal{O}$, and adequacy/binding assumptions H, establish*

$$H \wedge D(s,\theta) \wedge C_\theta(s) = \mathrm{ok}(a) \wedge \mathrm{Accepted}(s,\theta,a) \\ \Longrightarrow \mathrm{Refines}_{\mathcal{O}}(s,\theta,b_{\mathrm{init}},b_{\mathrm{run}}).$$

The relation must connect successful deployment to the exact installed runtime, and quantify over the admitted calls, worlds, and gas budgets. Unsupported inputs, validator rejection, and exhausted proof resources provide no such conclusion. For universal implementation correctness, remove the per-artifact acceptance premise only after proving the full admitted pipeline.

The repository does not yet discharge Obligation 1 for general Solidity. Selected artifact certificates do instantiate narrower versions. An implementation route would additionally need adjacent-stage simulation relations R_i and proofs that every successful stage maps R_i-related inputs to R_{i+1}-related outputs, with explicit failure behavior. A verified optimizer subroutine cannot supply the missing frontend, backend, or request composition proofs.

III. Two complementary proof routes

A. Implementation modeling and extraction

The Zig-to-Lean translation machinery is included. There are two families. First, `extract_*.py` parses restricted production source patterns and emits Lean programs or data. For example, `extract_compiler.py` admits matching and graph-construction syntax, while `extract_cache.py` binds key equality and normalization decisions. The lexer rejects unrecognized tokens, and the parsers constrain declarations, types, control flow, and recognized primitive calls. Many adapters intentionally demand exact source shapes. This is conservative but expensive to maintain when the Zig code changes.

Second, `lean/zig_air/` contains toolchain patch/build scripts, `SemanticExport.zig`, `SemanticValidity.zig`, capture and specialization linking tools, and scalar, aggregate, memory, and operation-specific generators. The toolchain builder pins Zig revision `7647adab80dd088f4de3610fd245915a912eb6ad` and version `0.17.0`. Capture occurs after semantic analysis and before legalization. The admitted scalar configuration requires ReleaseFast and LLVM; the inspected memory decoder requires AArch64 macOS, 64-bit pointers, and little-endian layout [3].

This is a restricted translation framework, not a general Zig compiler to Lean and not a verified Zig frontend. A normal installed Zig binary is insufficient for new captures: the instrumented toolchain, admitted source closure, capture records, and matching identities are required. Committed generated Lean models can be examined or built independently, without establishing fresh native capture or current source correspondence.

Obligation 2 (Translation and source safety)**.** *For an admitted Zig body z, capture u, and generated Lean program p, establish or explicitly trust the chain*

$$z \xrightarrow[\text{source identity}]{\text{admission/capture}} u \xrightarrow{\text{decode}} p,$$

and, under a representation relation R and source preconditions Q,

$$Q(n) \wedge R(n,m) \wedge n \Downarrow_z (r,n') \\ \Longrightarrow \exists m'.\ m \Downarrow_p (r,m') \wedge R(n',m').$$

Source undefined behavior and safety preconditions must be excluded or modeled; they cannot be recovered merely by proving the remaining AIR instructions.

The code is commendably explicit about this issue. Capture records require `sourceValidityConditionsPreserved` set to false; the validity sidecar describes an incomplete runtime-slice inventory. Memory generation restores selected guards at their anchors and rejects unsupported cases, including unadmitted length-based slicing. A proof about generated AIR is therefore not by itself a complete Zig source-safety theorem.

The typed source IR provides useful internal cross-checking. `Correspondence.lean` proves matcher, selection, and construction agreement between an interpreted IR and a separate readable-code backend. Selection is proved for arbitrary rule lists; construction preserves callback/error order for arbitrary monads. However, both backends share parsing and admission through `extract_compiler.py` and `compiler_ir.py`. Their agreement cannot detect a shared incorrect source interpretation [4].

B. Artifact certificates

Artifact validation instead proves a statement about a supplied source/output pair. Abstractly, for certificate π,

$$V(s,\theta,a,\pi) = \mathrm{accept} \Longrightarrow \mathrm{Refines}_{\mathcal{O}}(s,\theta,a).$$

The implementation need not be proved globally if the actual artifacts are bound to a sound certificate. Graph validation uses proved executable equivalence checks. Solidity bytecode validation uses Python to generate specialized Lean proof obligations and proofs about exact byte lists; the kernel, rather than a trusted Python success flag, checks their semantic content.

Source parsing and the association of source/settings/bytes with the theorem remain trusted admission responsibilities.

The shared artifact driver records exact request/response hashes, batches individual claims, creates a fresh package, compares preflight and compiled declaration audits, and invokes imported kernel replay. Corpus controls distinguish semantic rejection from resource failure. These are strong engineering choices. They still do not prove that a supplied graph snapshot contains every native entry, or that the trusted source parser has interpreted all admitted Solidity syntax correctly [5].

IV. Present proof obligations and results

A. Words, rewrites, and graph invariants

Let $\mathbb{W} = \mathrm{BitVec}(256)$; arithmetic wraps modulo 2^{256} where the model specifies EVM word operations. Checked Solidity arithmetic is a separate layer that can produce a panic outcome. Let a graph be $G = (N, K)$, where N maps 32-bit node identifiers to represented items and K maps expression keys to representatives. A key records item kind, opcode, data, argument identifiers, and sequence number. An environment ρ maps free word variables to $\mathbb{W}$.

Obligation 3 (Rewrite soundness)**.** *For every admitted rule r, facts f, and operands whose values satisfy those facts, prove*

$$\mathrm{matches}(r, f) \Longrightarrow [\![\mathrm{replace}(r)]\!]_\rho = [\![\mathrm{original}(r)]\!]_\rho.$$

Selection, construction, instruction interpretation, and graph updates must preserve the same equality; proving a rule table alone is insufficient.

The 45-entry production algebraic table has `production_rules_sound`, `production_selection_sound`, and `production_expression_sound` proof sources [6]. The extraction development extends beyond that table into constant folding, shifts, masks, nested operations, reassociation, dispatch ordering, and graph construction. Generalized meanings and callback interfaces allow modular proofs, but their hypotheses must be instantiated with the production behavior.

Write $\mathrm{Inv}(G)$ for cache/representation validity and $G \preceq G'$ for preservation of all previously represented values. For a request q with value v, the key recursive obligation is

$$\begin{aligned}\mathrm{Inv}(G) \wedge [\![q]\!]_G = v \wedge \mathrm{run}(q, G) = (r, G') \wedge \mathrm{Ord}(r) \\ \Longrightarrow \mathrm{Inv}(G') \wedge G \preceq G' \\ \wedge \big(\forall i,\ r = \mathrm{ok}(i) \Longrightarrow G'.\mathrm{read}(i) = v\big).\end{aligned}$$

`InsertionSortNormalization.lean` supplies `detailed_verified` for finite recursive model execution, including stable/extended graphs and closed local ownership. It assumes graph primitive contracts and frames, and quantifies over finite depth. It does not establish termination of unrestricted native recursion [7].

B. Normalization, ownership, and physical execution

Let m denote typed memory, a the logical argument list, and $\mathrm{Rep}(m, v, a)$ a represented view. A session also includes ownership registries and external effect state. The normalization function is

$$\mathrm{norm}(c, a) = \begin{cases} \mathrm{sort}(a) & c \wedge |a| \geq 2, \\ a & \text{otherwise.} \end{cases}$$

The proof must follow one execution through borrowing, scratch copying, allocation, sorting, transfer, and release, rather than combining unrelated final states.

Obligation 4 (Normalization lifecycle)**.** *From valid represented input, the actual fresh caller scratch allocation, and admitted external allocation/copy/free contracts, prove: exact normalized contents; source and unrelated-storage framing on the specified ordinary outcomes; independent ownership for borrowed duplication; identity-preserving transfer for owned temporaries; and release exactly by the current owner. OOM and incomplete/fault observations must retain the actual reached state.*

`NormalizationMemoryCaller.lean`, `Normalization.production_verified`, composes these obligations and derives scratch separation from an actual modeled allocation of two node IDs. Transferred contents preserve word meaning. Its statement explicitly allows preparation incompleteness or modeled memory exhaustion in its framing disjunction; it must not be paraphrased as unconditional successful cleanup. `NormalizationMemorySettles.lean` supplies the companion settling result. The external allocator, layout, copy, free, and stack contracts remain fixed boundary assumptions, not proved implementations [8].

Physical caller proofs deliberately distinguish ordinary results from rejected semantic observations. In `InternerOwnedMemory.lean`, ordinary results are success or language OOM/capacity/invalid-ID/assembly-exception errors. In `InternerPhysicalMeaning.lean`,

$$\begin{aligned}\mathrm{Ensures}(G, r, G', Q) := \\ \mathrm{Ord}(r) \Rightarrow \big(\mathrm{Inv}(G') \wedge G \preceq G' \\ \wedge (\forall v,\ r = \mathrm{ok}(v) \Rightarrow Q(v, G'))\big).\end{aligned}$$

Thus rejected observations do not prove memory safety or useful completion. `Caller.Verified` additionally preserves the exact body result and fault state, and closes ownership for ordinary outcomes. This is a sensible partial correctness interface; a deployment-facing theorem still needs admission and progress arguments to exclude pervasive rejection.

The general `Recursive.production_verified` composition retains explicit memory/semantic graph contracts, constructor coupling, constructor soundness, query coherence, graph validity, and represented input. More recent `NativeRecursion.production_find_verified` and `production_force_verified` specialize actual modeled callers and remove important callback premises. Find still requires native graph `Facts`, enclosing frames, query

agreements, source privacy and array representation, item representation, scratch allocation, and semantic validity. Force still requires the equality it is being asked to record; recording that equality does not prove it [9].

C. Native memory, hashing, and map operations

The AIR development contains typed memory and byte views, allocation and lifetime properties, read/write framing, stack cleanup, slice guards, finite-control/machine correspondence, loop reasoning, and concrete clients such as word copying/swapping and exponentiation. For example, `Control.Machine.finite_run` gives a sufficient fuel witness for finite code, and `finite_result` identifies every completed finite-machine result with the evaluator result. These are valuable reusable proofs, without implying adequacy of every translated Zig operation [10].

`WyhashUpdateAirComplete.lean`, theorem `invocation`, is substantial: under represented bytes, separation, geometry, arithmetic bounds, and the update invariant, some fuel completes captured update execution. Success matches the cell update model and preserves the hash invariant; OOM retains the reached write prefix; cleanup frames caller storage. It is not a theorem that the entire production hash producer, including all call preparation and finalization, has been connected to native lookup/publication.

There is a concrete adapter counterexample in `ExpressionMapHashNativeEmpty.lean`. The theorem `empty_native_update` connects the actual empty argument view to the captured update call. The view produces `emptyBytes`; execution yields OOM or unsupported, while preserving caller storage. Successful cell-model execution on an empty list cannot discharge this call. This is an explicitly documented model/admission limitation, not evidence of a native compiler bug.

`ExpressionMapSearchMemory.lean` proves receiver-based lookup and probe/ publication under a table/probe invariant and caller custody. Its `hashes : Key -> BitVec 64` parameter identifies a remaining boundary at this interface. Other local hash proofs reduce that boundary, but do not eliminate the full native producer obligation. Hash collision resistance is not required for correctness: probing and full key equality must handle collisions. Migration of all live entries during rehash, installation in the real owner, retirement of old storage, and derivation of initial native invariants remain important composition work [11].

D. Graph artifact validation

For input graph G, output graph G', returned identifiers I, and captured cache bindings K, `Inventory.validate_sound` proves

$$\begin{aligned} V_f(G, G', I, K) = \text{true} \Longrightarrow \forall \rho,\ \exists L, R : \\ \text{evalGraph}(G, \rho) = L \wedge \text{evalGraph}(G', \rho) = R \\ \wedge\, \text{map}(R[\cdot]?, I) = \text{map}(\text{some}, L) \\ \wedge\, \big(\forall (k, i) \in K,\ \exists v : \\ [\![k]\!]_{R,\rho} = v \wedge R[i]? = \text{some}(v)\big). \end{aligned}$$

The theorem also checks otherwise unused supplied nodes and cache bindings. `Inventory.certify` derives selected-root preservation from this same inventory certificate. Sound normalization includes associative/commutative constant regrouping for ADD, MUL, AND, OR, and XOR. Fuel can cause rejection; it does not justify an equality. Snapshot completeness, future mutations, native ownership, and arbitrary effectful graphs are outside this statement. There are separate graph-failure and raw word-block validator proofs [12].

E. Solidity deployment and call certificates

The call certificate admits inputs with calldata length below 2^{64}, an explicit proof domain rather than a protocol limit. The result includes short calldata, unknown selectors, nonpayable rejection, and trailing bytes. For an admitted source s, bytes b, and bound n, `SolidityCall.certify` consumes an inhabited ABI domain and

$$\forall x \in D,\ \exists c : \quad \text{run}(b, x, n) = \text{finished}(c, \text{outcome}_s(x)).$$

It derives metered execution for every fuel $f \geq n$ and gas g: execution finishes with $g - c$ if $c \leq g$, otherwise it is exceptional. Frame theorems include incoming-value rollback on failure. An inhabited ABI domain does not mean that the body ever succeeds; checked arithmetic may always panic.

Contract-level proofs compose all admitted selectors and rejection paths from the same runtime entry. Deployment proofs bind creation bytes, runtime bytes, installation, code-size/prefix restrictions, code-deposit gas, and subsequent calls. The storage route adds slot-layout, Boolean packing, supported initializers/constructors, static-write faults, rollback, warmth, and refund effects. The statement of `TracePreserves` quantifies over arbitrary admitted initial storage environments; it is not a finite test of storage values [13].

These are meaningful whole-artifact results for the admitted units, including selected empty/no-return/raw-word and typed-literal extensions. They do not cover arbitrary Solidity statements, loops, calls, inheritance, dynamic ABI values, or all deployment/linking forms. Transaction admission, fee settlement, and general creation-address behavior remain outside the modeled frame. The trusted source parsers and partial EVM specifications are therefore especially consequential correctness boundaries.

V. Soundness and Global Implementation Quality

A. What is well designed

The central architecture is credible. Proofs frequently quantify over arbitrary operands, memories, callback results, and gas budgets, rather than just proving closed examples. The development keeps exact reached failure states, derives scratch separation, distinguishes borrowing from ownership transfer, and relates memory and word semantics on the same execution. Strict unsupported outcomes are preferable to silently treating absent semantics as success. Explicit ABI witnesses and full dispatch partitions reduce vacuous or entry-point-only claims.

TABLE I
REPRESENTATIVE ENDPOINTS. P: PROOF SOURCE PRESENT FOR THE SCOPED RESULT; C: COMPOSITION WITH SUBSTANTIVE COMPILER PREMISES; O: OPEN ENDPOINT. NO ROW CLAIMS A FRESH BUILD OR REPLAY BY THIS REVIEW.

Boundary	Status	Present result	Remaining restriction or obligation
Algebraic table / selection	P	Word equalities for the production rule table and selected replacements	Source admission, production meaning and graph construction connection
Normalization lifecycle	P	Represented contents, ownership, transfer, framing, stated failures	Host contracts; valid caller input; no whole-optimizer conclusion
Recursive logical optimizer	C	Finite model invariants, old values and local ownership	Primitive contracts and frames; native admission/completion
Physical/native find and force	C	Same modeled caller execution, cleanup and word properties	Initial facts, representation, query agreement, equality for force, production wiring
AIR memory / hash update	P	Typed-memory laws and complete admitted update invocation	Source adequacy, empty-slice adapter, full native hash composition
Native expression map	C	Receiver lookup, probing and selected publication	Producer correspondence, rehash/install/retire and invariant establishment
Graph / word-block artifacts	P	Sound acceptance of exact supplied graph or byte block	Complete capture, restricted pure semantics and successful validation
Solidity artifacts	P	Restricted creation/runtime, dispatch, arithmetic and storage certificates	Trusted source admission/model adequacy; unsupported language constructs
General compiler	O	No complete endpoint theorem found; map explicitly open	Frontend, Yul passes, backend, linking and request composition, or broader artifact route

The audit and fresh-package infrastructure is stronger than relying on `lake build` alone. Source/tool digests, strict JSON admission, compiled theorem statement capture, logical-axiom closure, corruption controls, and fresh imported replay target different failure modes. The README correctly states that a build alone proves neither native freshness nor complete compiler verification. The existing boundary and map also acknowledge the major gaps.

No explicit logical escape hatch or demonstrated contradiction was found in the inspected proof sources. This is a source-review conclusion with the limits stated above, not an independent proof of soundness of the entire development. Logical validity, semantic adequacy, extraction correctness, and coverage are separate questions.

B. High-priority findings

a) F1: Production closure remains incomplete.: The native caller theorems are progress beyond abstract callback proofs, but their initial graph facts, input representation, and query contracts still need derivation from real compiler creation and call sites. The map leaves `optimizer.recursiveProduction` and the general pipeline open. Promoting a theorem named `production_verified` to a whole-compiler claim would overstate its type. The corrective action is a premise ledger tied to the actual final consumer, with initial-state establishment and preservation through every reachable mutation.

b) F2: A concrete native adapter path is unsupported.: The empty-slice/update mismatch above prevents substituting a successful local hash model for the native call sequence. It should be fixed and consumed at the hash/map endpoint before further claims of native hash completion. Acceptance should include empty input without assuming a successful update, and distinguish modeled rejection from a Zig runtime defect.

c) F3: Semantic adequacy and source safety remain trusted.: The independent Solidity/EVM models, restricted source parsers, AIR exporter, and source-validity restoration are real trust obligations. Hashes prove identity, not meaning. Dual generated backends share a frontend. Post-Sema ReleaseFast AIR does not retain all source safety conditions. The next trust reduction should target an independent account of an actually admitted semantic boundary, rather than merely adding more agreement proofs between models with the same origin.

d) F4: Conditional safety needs admission and progress.: The ordinary-result implication intentionally grants no semantic postcondition on unsupported or incomplete observations. Finite recursion unfolds can be correct while never establishing completion of an unrestricted native call. For each published endpoint, record separately: partial correctness, precondition establishment, exclusion of semantic faults, and sufficient fuel or termination. Successful examples are useful non-vacuity controls but cannot replace the quantified obligations. Likewise, a graph certificate cannot detect entries omitted before capture.

C. Maintainability and prioritization findings

a) F5: Proof effort is concentrated below the main semantic endpoint.: The extraction and AIR trees dominate the development while the artifact validation tree is comparatively small. This is not evidence that the proofs are poor: ownership and real failure behavior are difficult and valuable. It does indicate a prioritization risk. Many narrowly factored modules, large theorem telescopes, effect projections, and overlapping logical, cell-level, typed-memory, and native views increase the cost of seeing which remaining premise prevents deployment assurance. The empty-slice mismatch illustrates the cost of composition across these views.

Retain useful abstractions, but expose a small set of canonical endpoint interfaces. Group coherent invariants into explicit structures without hiding their fields; require every added lemma to identify the consumer premise it removes. Generate a dependency/assumption summary from declarations and retain non-vacuity fixtures beside the endpoint. Do not refactor all 1,440 files at once merely to reduce the file count.

b) F6: Evidence navigation is weaker than the proof infrastructure.: The coverage map records scoped obligations, but a reader still needs a compact view of the final theorem statements and their unclosed premises. Public acceptance evidence must identify the exact source revision, tool versions, theorem statements, domain restrictions, unresolved compiler premises, and retained replay artifacts. The preview release procedure describes how to retain and check commit-bound receipts; the existence of that procedure is not evidence that its gates have passed. Development checks should remain clearly distinguished from artifact certification; `prove.py` already makes that distinction.

VI. Is this the best approach?

There is no unconditional best route independent of the assurance target. For correctness of emitted Solidity artifacts, retain the hybrid architecture but make artifact certification the primary breadth strategy. It can cover many compiler transformations at once and avoids proving the native hash table, allocator protocol, and every optimizer implementation merely to establish a particular source/output semantic relation. Proof generation may remain untrusted if acceptance checks the exact statement and binding. Rejection is an acceptable loss of completeness, provided it is explicit.

For compiler process memory safety, corruption resistance, and all executions of a reusable optimization pass, artifact validation is insufficient. Implementation refinement is the right complementary route. Finish a bounded vertical slice from real initialization through a native caller and its mutations to the semantic endpoint. Continue low-level work when it discharges a named premise of that slice, especially the empty adapter path, native hash producer, and rehash/owner transition. Avoid making complete standard-library or allocator verification a prerequisite unless reducing that trust boundary is itself the chosen objective.

A verified general Zig frontend would reduce translation trust but introduces a much larger project. Reimplementing the optimizer in Lean could simplify some proof interfaces, but would require a new production integration and runtime trust story. Neither is justified as an immediate replacement by the inspected sources. The existing restricted translation should instead publish precise supported constructs, source-validity conditions, platform pins, and an explicit rejection inventory.

The next major semantic challenge for artifact coverage is controlled expansion into loops and interactions, with proof-producing symbolic execution or inductive invariants and an explicit policy for gas-sensitive behavior. A bounded unrolling that proves only selected executions must not be advertised as all-input preservation. Progress should be reported in admitted language features and endpoint premises discharged, not theorem totals.

VII. Review conclusion and reproducibility

The project has substantial, carefully scoped proof engineering and a defensible separation of implementation and artifact assurance. The strongest current claim supported by this review is the presence of conditional implementation proofs and soundness/certificate proof sources for explicitly restricted artifacts. General compiler correctness, universal native optimizer admission, and complete native map/hash composition remain open.

The accompanying `lean-verification-next-steps.md` gives ordered milestones and acceptance criteria. It proposes work; it changes no verification status. This paper uses the IEEEtran conference class and can be compiled with the system Tectonic installation:

```
tectonic -outdir docs
  docs/lean-verification-review.tex
```

Only document compilation was executed for this review. Lean elaboration, kernel replay, extraction freshness, native tests, and benchmark results must be obtained separately before asserting current acceptance.

References

[1] oksolc sources, `docs/compiler-verification-map.json`; `docs/compiler-verification-boundary.md`; `lean/README.md`.

[2] `lean/Contract/Check/Declarations.lean`, especially `closure` and `axioms`; `lean/Contract/Check/Axioms.lean`; `lean/prove.py`.

[3] `lean/zig_air/README.md`; `lean/zig_air/build_toolchain.py`; `lean/zig_air/capture.py`; `lean/zig_air/generate.py`; `lean/zig_air/generate_memory.py`; `lean/zig_air/SemanticExport.zig`; `lean/zig_air/SemanticValidity.zig`.

[4] `lean/extract_compiler.py`; `lean/extract_cache.py`; `lean/compiler_ir.py`; `lean/Compiler/Extraction/Correspondence.lean`.

[5] `test/lean/compiler_artifacts.py`; `test/lean/check_compiler_contracts.py`; `test/lean/solidity_function_source.py`; `test/lean/solidity_contract_source.py`.

[6] `lean/Compiler/Algebraic/Rules.lean`; `lean/Compiler/Algebraic/Proofs.lean`; `lean/Compiler/Extraction/DispatchProofs.lean`.

[7] `lean/Compiler/Extraction/InsertionSortNormalization.lean`; `lean/Compiler/Extraction/InternerPhysicalDispatchProduction.lean`; `lean/Compiler/Extraction/InternerPhysicalDispatchPreservation.lean`.

[8] `lean/Compiler/Extraction/NormalizationMemoryCaller.lean`; `lean/Compiler/Extraction/NormalizationMemorySettles.lean`; `lean/Compiler/Extraction/InternerCallerProduction.lean`; `lean/Compiler/Extraction/InternerPhysicalMeaning.lean`; `lean/Compiler/Extraction/InternerOwnedMemory.lean`.

[9] `lean/Compiler/Extraction/InternerNativeFindProduction.lean`; `lean/Compiler/Extraction/InternerNativeForceProduction.lean`; `lean/Compiler/Extraction/InternerNativeGraph.lean`.

[10] `lean/Compiler/ZigAir/MachineRefinement.lean`; `lean/Compiler/ZigAir/ControlSafety.lean`; `lean/Compiler/ZigAir/WyhashUpdateAirComplete.lean`.

[11] lean/Compiler/Extraction/ExpressionMapHashNativeEmpty.lean; lean/Compiler/Extraction/ExpressionMapSearchMemory.lean; lean/Compiler/Extraction/ExpressionMapWyhash.lean.

[12] lean/Compiler/Validation/GraphInventory.lean; lean/Compiler/Validation/Normalize.lean; lean/Compiler/Validation/GraphFailure.lean; lean/Compiler/Validation/WordBlock.lean.

[13] lean/Compiler/Validation/SolidityCall.lean; lean/Compiler/Validation/SolidityContract.lean; lean/Compiler/Validation/SolidityStorage.lean; lean/Compiler/Validation/SolidityStorageDeployment.lean; lean/Contract/Evm/Subset.lean; lean/Contract/Evm/Storage.lean; lean/Contract/Evm/Frames.lean; lean/Contract/Evm/StorageFrames.lean.